

Introducción a Uppaal Model Checker

Ignacio Facello, Manuel Nieto, Ignacio Tomas Ramirez

FAMAF

Universidad Nacional de Córdoba

<https://www.famaf.unc.edu.ar/>

{ignacio.facello, manuel.nieto, ignaciotramirez}@mi.unc.edu.ar

Resumen Este trabajo tiene como objetivo ser una introducción al *model checker* Uppaal. Se cubren tanto elementos históricos como aspectos técnicos de la herramienta, utilizando casos de estudio previos y un ejemplo particular para presentar su utilidad. Además, realizamos una comparación con otras herramientas similares y presentamos nuestras conclusiones.

Keywords: Uppaal · model checker · autómatas temporizados · LTL · verificación

1. Introducción

Uppaal es una herramienta para el modelado, simulación y verificación de sistemas en tiempo real utilizando autómatas temporizados, desarrollada en conjunto por el Departamento de Tecnologías de la Información de la Universidad de Uppsala y el Departamento de Ciencias de la Computación de la Universidad de Aalborg [1]. Su primera versión fue lanzada en 1995, y desde entonces ha estado en constante desarrollo. Actualmente, se encuentra presente tanto en entornos académicos como industriales, con la distinción de que su uso es gratuito solo para entornos académicos, mientras que cualquier otro tipo de uso requiere de una licencia de pago. Fue utilizada exitosamente para verificar desde protocolos de comunicación hasta aplicaciones multimedia.

En la herramienta existen extensiones que agregan diversas funcionalidades, entre ellas destacamos a Uppaal SMC y Stratego. La primera se enfoca en permitir model checking probabilístico y la segunda en generar, optimizar y comparar estrategias en juegos temporizados estocásticos con costos. Sin embargo, están por fuera del alcance de este trabajo, cuyo objetivo es introducir al lector a Uppaal y así permitirle evaluar si la herramienta satisface sus necesidades. Para ello, exploramos primero su interfaz gráfica y luego sus aspectos técnicos fundamentales. También presentamos distintos casos en los que se ha usado la herramienta y desarrollamos en mayor profundidad el ejemplo oficial de 2doors. Finalmente, analizamos cómo se compara Uppaal con otras alternativas disponibles.

2. Descripción de la herramienta

Uppaal posee una interfaz gráfica dividida en cuatro pestañas principales: *Editor*, *Symbolic Simulator*, *Concrete Simulator* y *Verifier* [2]. En conjunto, proveen funcionalidades para definir y manipular modelos de sistemas, correr simulaciones sobre los mismos, visualizar trazas de ejecución y describir propiedades a verificar. La especificación del sistema se realiza mediante autómatas utilizando una interfaz gráfica que permite definir estados como círculos y conectarlos mediante transiciones representadas por flechas. Además, se pueden asignar nombres e invariantes a los estados, así como guardas y actualizaciones a las transiciones. Adicionalmente, los modelos pueden incorporar constructores típicos de lenguajes de programación imperativos, incluyendo declaraciones de variables, funciones, bucles y tipos, entre otros, los cuales pueden ser definidos tanto en ámbito global como local.

Dado que Uppaal trabaja sobre autómatas temporizados, ofrece dos tipos de simulaciones: simbólica y concreta. La simulación simbólica permite examinar posibles trazas de ejecución del sistema mediante rangos de tiempo en los que los eventos pueden ocurrir, siendo particularmente útil para detectar fallas en etapas tempranas del diseño. Por otro lado, la simulación concreta muestra valores temporales precisos (en segundos) para ejecuciones particulares, visualizando los valores de los relojes internos del sistema. Complementando estas capacidades, el verificador de Uppaal permite, mediante una sintaxis similar a LTL, verificar propiedades de *safety*, *liveness* y *deadlocks*. Cuando alguna de estas propiedades no se cumple, el sistema genera un contra-ejemplo en forma de traza de ejecución, facilitando así el análisis y depuración del modelo.

3. Aspectos Técnicos

Uppaal explora los espacios de estados mediante una representación simbólica, en la cual cada estado del sistema se codifica como un vector de ubicaciones acompañado por una zona simbólica que describe las restricciones entre relojes. Estas zonas se representan internamente mediante matrices de cotas diferenciales (*Difference Bound Matrices*, DBM) las cuales permiten una exploración completa del espacio de estados a costa de un alto consumo de memoria [4]. Para complementar, Uppaal ofrece una estructura alternativa más compacta, pensada para reducir el consumo de memoria en modelos con muchos relojes, sacrificando la velocidad del chequeo [3].

Para mejorar el rendimiento en escenarios complejos, Uppaal también permite el uso de técnicas de aproximación que modifican el tratamiento del espacio de estados. La sub-aproximación consiste en almacenar únicamente un hash por estado alcanzado en lugar de la zona completa. Esto permite reducir significativamente la memoria requerida, pero puede omitir partes del espacio alcanzable y conducir a falsos negativos. Por otro lado, la sobre-aproximación relaja las restricciones temporales aplicando una envolvente convexa a las zonas, lo que introduce estados adicionales que no corresponden a comportamientos reales del

modelo. Esta técnica puede acelerar la verificación al reducir la fragmentación del espacio simbólico, pero también puede producir trazas espurias y falsos positivos. Cuando se utilizan estas formas de abstracción, el verificador puede emitir una respuesta indeterminada (`maybe satisfied`), lo que implica que no es posible confirmar ni refutar la propiedad debido a la pérdida de precisión [3].

El control sobre el almacenamiento de estados también es una herramienta clave en la mitigación de la explosión del espacio de estados. Uppaal ofrece varios modos de exploración que determinan qué estados conservar durante la verificación. El modo *None* guarda todos los estados alcanzables, asegurando una búsqueda completa. El modo *Conservative* excluye aquellos que contienen localizaciones asignadas (`commit locations`), lo que restringe los posibles *interleavings* y acelera el análisis. Finalmente, el modo *Aggressive* almacena sólo un estado por ciclo de reloj, lo que reduce notablemente el tamaño del grafo de exploración aunque limita la exhaustividad de la búsqueda [3].

Adicionalmente, Uppaal incorpora técnicas complementarias que mejoran la eficiencia del verificador sin afectar la corrección del resultado. Una de ellas es la reducción por simetría, que evita exploraciones redundantes. Otra técnica destacada es el método de línea de barrido, el cual elimina aquellos estados que ya no son necesarios si se define una medida de progreso global que garantiza que algunos de estos ya no serán necesarios [5].

Finalmente, para garantizar la terminación del análisis en presencia de relojes que pueden crecer indefinidamente, Uppaal aplica una abstracción automática conocida como extrapolación. Este mecanismo recorta el dominio de los relojes más allá de ciertos umbrales definidos teóricamente, asegurando así que el espacio simbólico permanezca finito. Existen distintas variantes de extrapolación como *Difference*, *Local*, *Upper/Lower*, entre otras. El verificador selecciona de forma automática la más permisiva que no afecte la validez de la propiedad a verificar [3].

4. Casos de Estudio

Uppaal ha sido aplicado en múltiples contextos, su página oficial presenta varios casos de estudio destacados. En particular resaltamos:

Real-Time SCADA Attack Detection by means of Formal Methods [6], investigación llevada a cabo por el IIT (*Istituto di Informatica e Telematica*) y la universidad de Molise. Se busca verificar un método para la detección de ataques a sistemas *SCADA* para control de infraestructura crítica. El trabajo logra confirmar la habilidad del método propuesto para discernir entre ataques reales y comportamiento normal.

Bang & Olufsen Audio/Video Protocol [7], una colaboración entre la empresa Bang & Olufsen, BRICS (*Basic Research in Computer Science*) y la universidad de Aalborg. Se conoce la presencia de un error de temporización en un protocolo de comunicación que ocasionalmente perdía mensajes, el objetivo del trabajo es identificarlo y resolverlo. Se logra identificar que la fuente se encuentra en

la detección de colisiones (desconexión prematura del mecanismo) y se propone una corrección validada formalmente.

Philips Audio Protocol with Bus Collision [8], por parte de la empresa Philips en colaboración con las universidades de Aalborg, Uppsala y Nijmegen. El objetivo es analizar un protocolo de comunicación para la capa física que conecta varios dispositivos de audio donde tenemos varios emisores y un único canal. Se logra confirmar que evita colisiones y se detectan las condiciones específicas de sincronización que pueden llevar a un deadlock.

Bounded Retransmission Protocol [10], nuevamente una colaboración entre Philips con las universidades Twente y Erlangen. Se busca verificar corrección en escenarios con pérdida de mensajes incluyendo temporización para un protocolo de retransmisión en uno de los productos de la marca. Los autores logran verificar un método para modelar canales complejos de forma simplificada y corregir supuestos erróneos del protocolo ajustando sus tiempos críticos.

TDMA Protocol Start-Up Mechanism [9], colaboración entre las universidades de Chalmers y Uppsala. Se presenta el protocolo DACAPO para TDMA (*Time Division Multiple Access*). Se verifica su confiabilidad para hasta 4 estaciones y se mide el tiempo máximo para la inicialización de TDMA bajo el mismo (21 ciclos).

Gearbox Controller [11], es un proyecto colaborativo entre Mecel AB (industria automotriz) y la universidad de Uppsala. Este caso se centra en el diseño y la verificación formal de un controlador de cambios de marcha para vehículos. Se verificaron 46 propiedades de seguridad, funcionalidad y tiempo de respuesta acotado en 2,99 segundos utilizando sólo 24MB de RAM.

5. Análisis de 2doors

Uppaal incluye varias demos que muestran las capacidades de la herramienta. A continuación veremos el caso particular de 2doors, el cual implementa un modelo de dos puertas con exclusión mutua. Una puerta comienza a abrirse cuando se presiona su botón siempre y cuando la otra no esté abriéndose o cerrándose. El proceso de apertura tarda seis segundos y la puerta permanece abierta durante un intervalo de entre cuatro y ocho segundos. Finalmente, le toma seis segundos cerrarse y debe mantenerse cerrada al menos cinco antes de poder volver a abrirse. Para implementar este sistema se utilizaron dos modelos distintos: **Door** y **Button**, ambos descritos en la Figura 1.

Antes de presentar la implementación, introducimos conceptos importantes de la herramienta que serán útiles para su comprensión: En los autómatas temporizados, los estados pueden estar anotados con un **invariante**, que se utiliza para restringir el tiempo mínimo o máximo que el sistema permanece en dicho estado. Los estados que tienen dos círculos representan el estado inicial del autómata. Las transiciones, por su parte, pueden estar condicionadas por una **guarda** que limita cuándo pueden ejecutarse, además de poder incluir una **actualización** para modificar variables y relojes. Finalmente, la sincronización entre modelos se realiza mediante **canales**: al anotar un par de transiciones con un canal (una

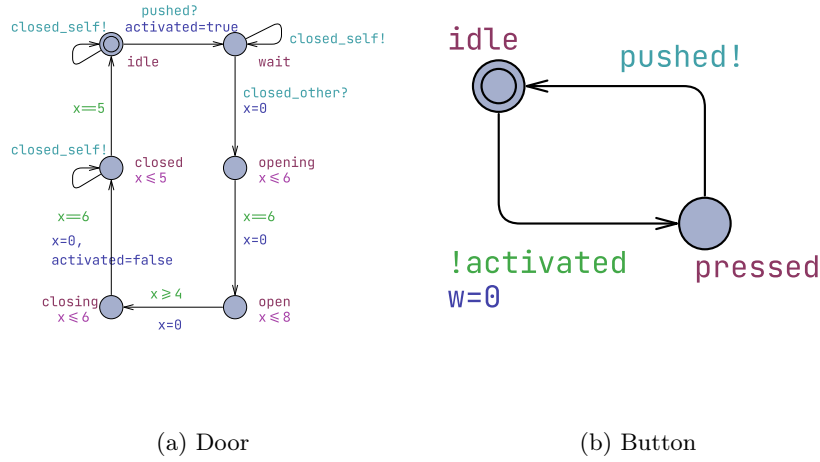


Figura 1: Los dos autómatas genéricos de nuestro modelo.

como emisora **canal!** y otra como receptora **canal?**), se indica que ambas deben estar habilitadas simultáneamente para que ocurra la sincronización. Cuando un canal está marcado como urgente, esta sincronización debe producirse inmediatamente, sin permitir el avance del tiempo.

Modelo Door El autómata comienza en el estado *idle* y permanece inactivo hasta que se recibe una señal a través del canal urgente *pushed*, enviada por el botón correspondiente. El autómata pasa al estado *wait*, permaneciendo a la espera de una señal proveniente del canal urgente *closed_other*, la cual indica que la otra puerta se encuentra cerrada. Si esto ocurre, el modelo transiciona al estado *opening* y reinicia su reloj interno *x*.

Notar el uso combinado de invariantes y guardas para especificar que la puerta tarda exactamente seis segundos en abrirse. Por un lado, la guarda no permite que se tome la transición a menos que el reloj sea igual a 6, pero no asegura que vaya a ser tomada, mientras que la invariante solo permite al autómata mantenerse en este estado hasta que el reloj sea menor o igual 6. Es decir, la transición siempre se toma cuando el reloj es exactamente 6. Al salir de este estado, la puerta se encuentra completamente abierta, luego se repite el truco anterior para especificar el rango de entre cuatro y ocho segundos que puede permanecer abierta. Por último, se inicia el proceso de cierre, que tarda otros seis segundos.

Finalmente, la puerta permanece cerrada durante cinco segundos durante los cuales no responde a pedidos de apertura por parte de su botón. Cabe aclarar que en los estados *idle*, *wait* y *closed*, el autómata puede emitir una señal

por el canal urgente `closed_self`, notificando a la otra puerta que se encuentra cerrada.

Modelo Button Representa un botón con dos estados, `idle` y `pressed`. Para pasar desde `idle` a `pressed` necesitamos que la variable booleana `activated` sea `false`, esto sirve para indicar que la puerta no está procesando un pedido de apertura. Al realizarse esta transición, se reinicia el reloj interno del modelo. Para volver al inicio necesitamos que su puerta pueda recibir el mensaje de `pushed`.

Verificación Esta demo además contiene algunas propiedades que el sistema debería cumplir: En primer lugar, buscamos garantizar exclusión mutua, asegurando que ambas puertas nunca estén abiertas al mismo tiempo. Esta propiedad se verifica con la fórmula LTL " $A[] \text{ not } (\text{DoorA.open and DoorB.open})$ ". También se debe garantizar liveness acotado, es decir, que tras presionar un botón, la puerta correspondiente comience a abrirse en un tiempo razonable, expresado como " $A[] (\text{DoorA.opening imply ButtonA.w} \leq 31)$ ". Se toma 31 segundos como cota máxima, ya que es el peor caso posible con los parámetros con los que está configurado el problema. Además, se comprueba que existe al menos una ejecución en la que cada puerta logra abrirse con " $E<> \text{DoorA.open}$ " y " $E<> \text{DoorB.open}$ ". Por último, se verifica que el sistema no presenta deadlock con " $A[] \text{ not deadlock}$ ". Todas estas propiedades pueden ser evaluadas con el verificador de Uppaal, permitiendo confirmar que el sistema es seguro, reactivo y funcional. En la Figura 2 se pueden ver todas las propiedades verificadas.

Overview	
$A[] \text{ not } (\text{DoorA.open and DoorB.open})$	●
$A[] (\text{DoorA.opening imply ButtonA.w} \leq 31) \text{ and } (\text{DoorB.opening imply ButtonB.w} \leq 31)$	●
$E<> \text{DoorA.open}$	●
$E<> \text{DoorB.open}$	●
$\text{DoorA.wait} \rightarrow \text{DoorA.open}$	●
$\text{DoorB.wait} \rightarrow \text{DoorB.open}$	●
$A[] \text{ not deadlock}$	●

Figura 2: Propiedades verificadas en el sistema.

6. Comparación con otros Model Checkers

Dado que existe una gran cantidad de Model Checkers disponibles y se podrían dedicar trabajos de investigación enteros a cada uno de ellos, hemos decidido limitar la comparación con Uppaal a tres de ellos: **LTSA**, **Prism** y **Spin**. Separamos la comparación en varios criterios:

Lenguaje de modelado y expresividad Las cuatro herramientas utilizan formalismos distintos para describir sus modelos. Mientras que las demás herramientas

utilizan un lenguaje propio para la especificación de modelos, Uppaal se distingue por permitir la construcción gráfica de autómatas. Además, la especificación de propiedades se hace mediante gramáticas libres de contexto que nos provee la documentación [12].

Capacidades de verificación Si bien difieren en la forma de hacerlo, todas proveen las verificaciones básicas que se esperan de un Model Checker, es decir, safety, liveness y deadlocks. Vale la pena mencionar que todas proveen contraejemplos en modo de trazas de ejecución al momento de verificar una propiedad que no se cumple en el modelo, siendo Uppaal y Spin las más descriptivas.

Exploración del espacio de estados Esta área impacta principalmente en el uso de recursos del sistema donde se realiza la verificación. Todas tienen una forma particular de manejar sus espacios de estados. Tanto Uppaal como Spin [13] y Prism [14] lo hacen de manera simbólica, lo que permite manejar modelos con gran cantidad de estados. Por otro lado, LTSA hace una exploración explícita, por lo que está limitada a sistemas pequeños. Spin utiliza una técnica llamada *supertrace* [13] la cual puede colapsar el espacio de estados a una poca cantidad de bits por estado alcanzable, volviéndola de gran utilidad para manejar grandes problemas de verificación.

Fortalezas distintivas Mientras LTSA destaca en la verificación intuitiva de sistemas concurrentes pequeños mediante autómatas finitos, Spin sobresale en la detección de errores sutiles en sistemas asíncronos mediante LTL. Uppaal domina el análisis de propiedades para autómatas temporizados, y Prism ofrece capacidades únicas para el razonamiento probabilístico y cuantitativo basado en cadenas de Markov.

Cabe aclarar que no se incluye una comparación de rendimiento entre las herramientas debido a la falta de literatura actualizada sobre el tema. Las métricas que existen sobre Uppaal en cuanto al uso de recursos fueron realizados sobre versiones antiguas de la herramienta y ejecutados en hardware actualmente obsoleto y consideramos que no reflejan su desempeño actual.

7. Conclusión

A lo largo de este trabajo hemos destacado características de la herramienta elegida, centrándonos en facilitar al lector decidir si Uppaal cumple con sus requerimientos. Uno de los principales puntos a favor son las pocas barreras para comenzar a utilizarla. Su instalación es sencilla y la documentación oficial resulta simple e intuitiva. Además, los ejemplos incluidos con la instalación son diversos, simples y representativos de las múltiples funcionalidades que Uppaal ofrece. Comprenden casos con múltiples niveles de complejidad que introducen orgánicamente al usuario al diseño y la verificación.

Consideramos que es una herramienta fundamental para la verificación formal de sistemas temporizados, destacándose por su simplicidad y expresividad a

la hora del modelado. Permite verificar exhaustivamente propiedades de safety, liveness y ausencia de deadlocks, tanto para ejemplos simples como en sistemas complejos. Los casos de estudio (incluyendo el modelo 2doors) demuestran gran aplicabilidad industrial y una curva de aprendizaje manejable gracias a su interfaz gráfica intuitiva para el diseño de autómatas y la generación automática de trazas de error que agilizan la depuración.

Al compararla con otras herramientas como Spin, Prism y LTSA se destacan sus fortalezas: Uppaal es superior en aplicaciones de tiempo real, como en sistemas de transporte, robótica o comunicaciones multimedia. Además, sus diversas opciones de análisis (sub/sobre-aproximación, exploración agresiva/conservativa, extrapolación) permiten modificar el proceso de verificación para mitigar explosiones de estados y así facilitar el procesamiento de grandes modelos.

A pesar de sus numerosas fortalezas, Uppaal presenta ciertas limitaciones que deben considerarse. Posee un lenguaje restringido para expresar las propiedades a verificar, basado en una versión limitada de la lógica temporal LTL que no permite el uso de operadores más expresivos como *weak-until*. Tampoco permite anidar operadores [16], por lo que solo podemos expresar propiedades de safety o liveness simples. Además, la herramienta enfrenta el problema de la explosión del espacio de estados, especialmente en modelos con múltiples relojes o componentes concurrentes, lo que puede afectar severamente el rendimiento y la escalabilidad. Esto se debe a que su versión académica no aprovecha el paralelismo de múltiples núcleos [15] por lo que incluso hardware moderno no acelera el proceso de verificación.

En síntesis, Uppaal es una opción amigable, versátil y poderosa para la verificación de sistemas temporizados. Aunque no logramos explorar la totalidad de la herramienta, futuros trabajos podrían profundizar en el uso de Uppaal SMC o Stratego, variantes especializadas que abordan problemas específicos. Asimismo, sería valioso complementar este análisis con estudios de rendimiento actualizados, incluyendo benchmarks de desempeño que reflejen la evolución de la herramienta sobre hardware moderno.

Referencias

1. Behrmann, G., David, A., Larsen, K.G.: A Tutorial on Uppaal 4.0. Department of Computer Science, Aalborg University, Tech. Rep. 1(1), p.1 (2008)
2. UPPAAL GUI Reference, <https://docs.uppaal.org/gui-reference/>, last accessed 2025/05/28
3. UPPAAL Documentation on Menu Options, <https://docs.uppaal.org/gui-reference/menu-bar/options/>, last accessed 2025/05/29
4. Mohammed, F., Marius, M., Maryline, Z.: Scalable Computation of Inter-Core Bounds Through Exact Abstractions, p.4 (2024)
5. Gerd, B., Alexandre, D., Kim, L.: A Tutorial on Uppaal 4.0, pp. 26-27, Aalborg, Denmark (2006)
6. Mercaldo, F., Martinelli, F., Santone, A.: Real-time SCADA attack detection by means of formal methods. 2019 IEEE 28th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE). pp. 231–236 (2019)

7. Havelund, K., Skou, A., Larsen, K.G., Lund, K.: Formal Modelling and Analysis of an Audio/Video Protocol: An Industrial Case Study Using Uppaal. In: Proceedings of the 18th IEEE Real-Time Systems Symposium, pp. 2–13. San Francisco, California, USA (1997)
8. Bengtsson, J. et al.: Verification of an Audio Protocol with Bus Collision Using UPPAAL. In: Proceedings of the 8th International Conference on Computer Aided Verification (CAV '96), pp. 244–256. Springer, Berlin, Heidelberg (1996)
9. Lönn, H., Pettersson, P.: Formal Verification of a TDMA Protocol Start-Up Mechanism. In: Proceedings of the 1997 IEEE Pacific Rim International Symposium on Fault-Tolerant Systems, pp. 235–242. Taipei, Taiwan (1997)
10. D'Argenio, P.R. et al.: The bounded retransmission protocol must be on time!. In: Brinksma, E. (eds) Tools and Algorithms for the Construction and Analysis of Systems. TACAS 1997. Lecture Notes in Computer Science, vol 1217. Springer, Berlin, Heidelberg (1997)
11. Lindahl, M., Pettersson, P., Yi, W.: Formal Design and Analysis of a Gear Controller. In: Steffen, B. (ed.) Proceedings of the 4th International Workshop on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 1998). LNCS, vol. 1384, pp. 281–297. Springer, Lisbon, Portugal (1998)
12. UPPAAL Documentation on Declarations. <https://docs.uppaal.org/gui-reference/system-editor/declarations/>, last accessed 2025/05/28
13. What is Spin?. <https://spinroot.com/spin/what.html>, last accessed 2025/05/28
14. PRISM Manual. https://www.prismmodelchecker.org/manual/FrequentlyAskedQuestions/PRISMModelling#max_model_size, last accessed 2025/05/28
15. Grobelna, I. et al.: A Systematic Review on the Applications of Uppaal. Sensors. p.28 (2025)
16. Samat, P.A. et al.: Analysis of the Model Checkers' Input Languages for Modeling Traffic Light Systems. Journal of Computer Science. pp. 225–233 (2011)